

Relatório técnico: Comparação entre arquiteturas MVC, MVP e MVVM integradas a backends REST e reativos.

Introdução

Este relatório apresenta uma análise crítica comparativa entre as arquiteturas de frontend MVC, MVP e MVVM, aplicadas a um mesmo aplicativo ToDo desenvolvido em React com TypeScript, integrado a um backend implementado na plataforma Supabase. O backend foi explorado em dois modos distintos: REST tradicional e reativo/event-driven. A análise baseia-se nas decisões de projeto, dificuldades encontradas e consequências práticas observadas durante a implementação do código, conforme exigido pelo enunciado do trabalho.

Onde ficou a maior parte da lógica em cada arquitetura?

No MVC, a lógica concentrou-se nos Controllers, acumulando chamadas ao Supabase, eventos e controle de estado, o que gerou acoplamento com a View e duplicação de lógica, tornando a arquitetura artificial no contexto do React.

No MVP, a lógica foi centralizada no Presenter, reduzindo o acoplamento da View, porém com aumento de verbosidade e complexidade à medida que o código evoluiu.

No MVVM, a lógica concentrou-se no ViewModel, responsável pelo estado e sua sincronização com o Supabase, resultando em fluxo de dados mais previsível e melhor adaptação ao React, especialmente com backend reativo.

Qual arquitetura foi mais simples de integrar com o backend reativo?

A integração com o backend reativo evidenciou diferenças significativas entre as arquiteturas. O MVVM apresentou a integração mais direta, pois o ViewModel já centralizava o estado da aplicação. Na prática, os eventos automáticos recebidos do Supabase puderam ser incorporados diretamente ao estado, eliminando a necessidade de requisições adicionais ou lógica de sincronização manual.

No MVP, a integração exigiu que o Presenter assumisse explicitamente o gerenciamento das subscriptions do backend reativo. Embora funcional, essa abordagem tornou o Presenter progressivamente mais complexo, acumulando responsabilidades relacionadas tanto ao fluxo de dados quanto ao ciclo de vida das conexões reativas.

No MVC, a integração foi a menos eficiente. Controllers passaram a acumular lógica de escuta de eventos, atualização de Model e notificação da View, o que agravou o acoplamento entre camadas.

O que mudou no frontend ao trocar REST por reativo?

A mudança de REST para reativo impactou o gerenciamento de estado, já que no REST as atualizações dependiam de requisições explícitas.

Com o backend reativo, essa limitação desapareceu, mas trouxe novas exigências ao frontend. O código precisou lidar com eventos assíncronos contínuos, o que expôs fragilidades arquiteturais, especialmente no MVC e no MVP. Arquiteturas que não centralizavam o estado exigiram lógica adicional para evitar inconsistências visuais e estados duplicados.

No MVVM, a mudança foi significativamente menos impactante, pois o ViewModel já funcionava como fonte única de verdade. Isso demonstrou, de forma prática, que arquiteturas orientadas a estado são mais adequadas para aplicações que consomem backends reativos.

Qual arquitetura você escolheria para um sistema maior? Por quê?

Com base na experiência prática obtida durante a implementação, a arquitetura MVVM mostrou-se a escolha mais adequada para sistemas de maior porte. Sua separação clara entre View e ViewModel reduziu o acoplamento, facilitou a introdução de novas funcionalidades e tornou o código mais resiliente a mudanças no backend.

O MVP, apesar de mais organizado que o MVC, apresentou crescimento rápido da complexidade do Presenter, indicando risco de se tornar um novo ponto de acoplamento em sistemas maiores. O MVC, por sua vez, demonstrou limitações estruturais relevantes no contexto do React, dificultando tanto a manutenção quanto a integração com arquiteturas reativas.

Assim, a escolha pelo MVVM não se baseia apenas em adequação teórica, mas na observação concreta de menor esforço de manutenção, maior clareza do fluxo de dados e melhor alinhamento com o paradigma reativo adotado no projeto.

Conclusão

A implementação prática das três arquiteturas evidenciou que decisões arquiteturais impactam diretamente a clareza do código, o gerenciamento de estado e a facilidade de manutenção. A experiência demonstrou que abordagens excessivamente teóricas, como o MVC aplicado ao React, tendem a gerar acoplamentos artificiais.

Por outro lado, arquiteturas alinhadas ao paradigma reativo e orientadas a estado, como o MVVM, mostraram-se mais adequadas para aplicações modernas, especialmente quando combinadas a backends event-driven. Dessa forma, o trabalho reforça que a escolha arquitetural deve ser guiada não apenas por definições conceituais, mas principalmente por evidências práticas obtidas durante a implementação.